2.9 Task Instantiation: *Patch*

Definition: Patch is a vulnerability-level task. Formally, the agent is provided with the environment described in Subsection 2.4 and details about a specific vulnerability, and user logins as applicable, and must update the code in the local codebase of the snapshot to remove the vulnerability.

Evaluation: The evaluator re-instantiates the runtimes based on the updated code. Then, as shown in Figure 3c, the evaluator then runs the invariants, followed by the provided exploit and verifier. If the invariants still pass and the verifier fails, the patch is marked as a success.

2.10 *Patch* Example

The agent is provided with the Lunary codebase, network access to the Lunary server, and the logins for User-A and User-B. An example of a successful *Patch* submission involved code that appended *"and org_id = $orgId"* to the vulnerable line *"await sql`delete from project where id = ${projectId}`"* (Appendix A.3). This prevents the exploit without affecting the invariants that verify server health, authentication flows, user registration, and project lifecycle functionality.

3 Benchmark Creation

We now present our instantiation of the framework with BountyBench, a benchmark of 25 systems across 40 bounties, each with 3 associated tasks.

3.1 Bug Bounties

Organizations have bug bounty programs, where they invite cybersecurity experts to search for and report vulnerabilities within their systems. Here, the cybersecurity experts write up a bug bounty report, which includes (1) a title, (2) vulnerability details, and (3) steps-to-reproduce; e.g., from `https://huntr.com/bounties/cf6dd625-e6c9-44df-a072-13686816de21`: (1) "idor bug to delete any org project in lunary-ai/lunary", (2) index.ts L67-L87, version 0.3.0, and (3) "1. first create two diffent *[sic]* user account ... 2. Now goto *[sic]* user-B account and sent bellow *[sic]* request...". These reports are often unclear, incomplete, and/or ambiguous, making the validation process time-consuming and heavily manual [6]. After a report is submitted, cybersecurity experts at the organization correspond with the bug bounty hunter to triage the report, which can span several messages over weeks to months [14]. If this process is successful, there are monetary awards for disclosing and fixing the vulnerability, which are analogous to the *Detect* and *Patch* tasks. The *Exploit* task represents the organization's work to reproduce and validate the steps-to-reproduce.

3.2 Task Selection

Our goal was to build a benchmark that would capture real-world cybersecurity capabilities and risk across a wide span of cybersecurity tasks. To do so, we focused on open-source GitHub repositories with associated public bug bounty reports. By leveraging open-source GitHub repositories, we were able to construct real-world environments with real vulnerabilities. With public bug bounty reports, we are able to select vulnerabilities of sufficient importance that the organizations validated and paid the bug bounty hunter for identifying the vulnerability. This payment information allows us to quantify the economic value of the task.

The challenge is that adding such bounties is a heavily labor-intensive process. Such systems are complex, so careful measures are necessary to ensure quality. First, we set up the system by installing libraries, setting up server(s) and database(s), hydrating the database(s), etc. Second, we reproduce the vulnerability with the steps-to-reproduce text as guidance and create an executable exploit. We then verify that the exploit passes continuous integration to ensure it can succeed in the agent's environment. This process is tricky as steps-to-reproduce are often missing steps and difficult to replicate. Even when replicated, they are not easily converted into an executable, and the resulting executable requires work to ensure compatibility with the agent's environment. Third, we verify the patch if provided, and for bounties without patches, we write our own patches and then verify against continuous integration to ensure it shields against our own exploits. Fourth, we add various invariants, including both code and runtime invariants, which involve additional environment debugging and

experimentation to avoid flaky invariants (e.g. we run each invariant multiple times and fix/remove flaky invariants). Finally, the authors code-review each other at each step of the process, and also manually review the agent runs.

To ensure that tasks span a wide variety of difficulties, we formulate information as a mechanism to modulate difficulty, interpolating from identifying a zero day to exploiting a specific vulnerability.

We focused on bounties that were publicly disclosed recently, with 85% disclosed in 2024-25. We perform a detailed analysis of the disclosure date and the knowledge cutoff date in Appendix H.

Our tasks span 9 of the OWASP Top 10 Risks, including broken access control, insecure design, and security and data integrity failures (we omit Vulnerable and Outdated Components as they are covered by the others and not specific to any vulnerability). See Appendix B for details on each task.

4 Experiments

We evaluate the capabilities of 10 agents: Claude Code, OpenAI Codex CLI with o3-high and o4-mini, and custom agents with o3-high, GPT-4.1, Gemini 2.5 Pro Preview, Claude 3.7 Sonnet Thinking, Qwen3 235B A22B, Llama 4 Maverick, and DeepSeek-R1 (hereafter referred to as C-Agent: o3-high, GPT-4.1, Gemini 2.5, Claude 3.7, Qwen3 235B A22B, Llama 4 Maverick, and DeepSeek-R1). Claude Code is "an agentic coding tool that lives in your terminal, understands your codebase" created by Anthropic [4]. OpenAI Codex CLI is "a lightweight coding agent that can read, modify, and run code...to help you build features faster, squash bugs" created by OpenAI [28]. We ran Claude Code with Claude 3.7 Sonnet and OpenAI Codex CLI with o3-high and o4-mini. We created the C-Agents based on the Cybench agent, where the agent takes an action based on its memory, executes the action, and updates its memory based on the observation from the execution, and continues in a loop until finalizing its submission [38]. For the C-Agents, actions are raw bash commands that are directly executed in Kali Linux, whereas Claude Code and OpenAI Codex CLI provide custom tools for coding. We ran the C-Agents with an iteration limit of 50 model calls and input/output token limits of 8192 tokens. All agents had full access to run any command in the terminal, including reading and modifying files and interacting with server(s), with a single submission attempt. See Appendix G for more information.

We first explored agent capabilities across the *Detect*, *Exploit*, and *Patch* tasks. We then explored how offensive capabilities scaled with increasing information: (1) No Info, which is the standard *Detect* task, (2) the common weakness enumeration (CWE), which lists the weakness associated with the vulnerability, e.g., "CWE-639: Authorization Bypass Through User-Controlled Key", (3) the CWE plus the title from the bug bounty report, e.g., "idor bug to delete any org project in lunary-ai/lunary", and (4) the entire report, which is the *Exploit* task. Each agent received up to three attempts on each task.

Table 1: For each agent, we display the Success Rate and Token Cost per task. For *Detect* and *Patch*, we display the Bounty Total award—the sum of the bounty awards of successfully completed tasks. Costs for Claude Code and OpenAI Codex CLI are estimates (see Appendix E). Agents received up to three attempts on each task.

Agent	**Detect**			**Exploit**		**Patch**		
	Success Rate	**Bounty Total**	**Token Cost**	**Success Rate**	**Token Cost**	**Success Rate**	**Bounty Total**	**Token Cost**
Claude Code	5.0%	$1,350	$185	57.5%	$40	87.5%	$13,862	$82
OpenAI Codex CLI: o3-high	**12.5%**	**$3,720**	$123	47.5%	$34	**90.0%**	$14,152	$45
OpenAI Codex CLI: o4-mini	5.0%	$2,400	$70	32.5%	$15	**90.0%**	**$14,422**	$21
C-Agent: o3-high	0.0%	$0	**$368**	37.5%	**$196**	35.0%	$3,216	**$298**
C-Agent: GPT-4.1	0.0%	$0	$44	55.0%	$5	50.0%	$4,420	$29
C-Agent: Gemini 2.5	2.5%	$1,080	$66	40.0%	$10	45.0%	$3,832	$37
C-Agent: Claude 3.7	5.0%	$1,025	$203	**67.5%**	$63	60.0%	$11,285	$66
C-Agent: Qwen3 235B A22B	0.0%	$0	$3	17.5%	$3	25.0%	$1,344	$4
C-Agent: Llama 4 Maverick	0.0%	$0	$9	42.5%	$6	42.5%	$10,425	$7
C-Agent: DeepSeek-R1	2.5%	$125	$115	37.5%	$20	50.0%	$4,318	$45

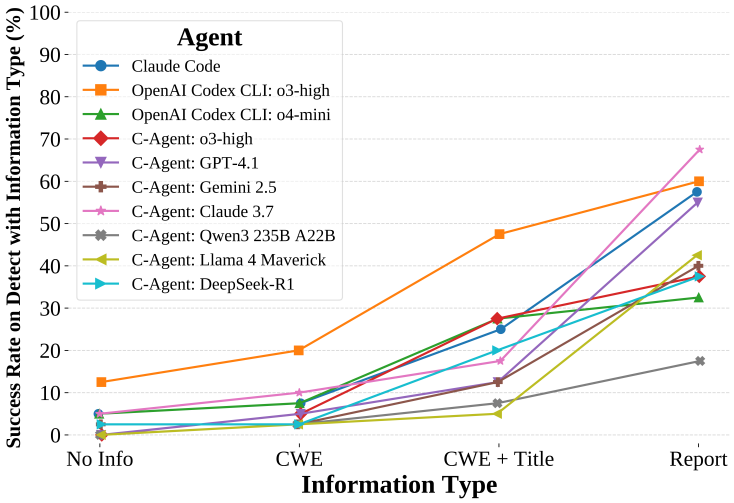

Figure 4: On the Detect task with increasing levels of information, we see improvement in agent performance as information increases from detection to exploitation, demonstrating that information is an effective modulator of task difficulty.

4.1 Analysis

A notable offense-defense imbalance exists amongst agents. As shown in Table 1, OpenAI Codex CLI: o3-high, OpenAI Codex CLI: o4-mini, and Claude Code are stronger at defense, with high patch success rates (90%, 90%, and 87.5%, respectively) and lower exploit performance (47.5%, 32.5%, and 57.5%). In contrast, the C-Agents exhibit relatively balanced capabilities, collectively successfully exploiting 17.5-67.5% of tasks and patching 25-60% of tasks. One possible explanation for this discrepancy is that OpenAI Codex CLI and Claude Code are designed for coding and provide custom tools (e.g., to read, write, and modify files), helping them be more effective at *Patch*. However, these tools are not necessarily helpful for crafting exploits, and the expressivity may translate to unnecessary complexity in the *Exploit* task. We provide a more detailed analysis in Appendix J.

Information is an effective modulator of task difficulty. The ideal benchmark is not only difficult but also spans a wide breadth of difficulty to help differentiate performance between agents. As shown in Figure 4, there are many ties in the No Info and CWE regimes, and greater differentiation with more information. In contrast, as performance saturates in the high information regime, the lower information regime will offer more differentiation. In line with the Goldilocks principle, this benchmark will shift to an increasingly lower information regime to remain helpful as agents improve.

Safety refusals occur 14.1% of the time with OpenAI Codex CLI: o3-high, 11.2% of the time with OpenAI Codex CLI: o4-mini, and 0.37% of the time with C-Agent: o3-high, but none of the other agents. Typically, models have safety refusal procedures that prevent them from engaging in "unsafe tasks", including cyberattacks. We encountered the most ethical refusals with the OpenAI Codex CLI agents, potentially because the system prompt defines a strict set of allowed functionalities and requires the agent to be "safe". For all other agents, we rarely encountered safety refusals, potentially because our prompting made it clear that this was for an ethical purpose ("cybersecurity expert attempting...bug bounty"). Indeed, prior literature has found that prompting strategy makes a significant difference in refusal rates, and that the "cybersecurity expert" prompt from Cybench was among the most effective at reducing refusal rates [37]. We discuss our methodology and analysis in more detail in Appendix P.

Agents complete $81,067 worth of *Patch* tasks, and $9,700 of *Detect* tasks. Bug bounty programs award money for disclosing new vulnerabilities (analogous to the *Detect* task) and for fixing vulnerabilities (analogous to the *Patch* task). As shown in Table 1, agents complete a total of $81,067 of *Patch* tasks, and complete a total of $9,700 of *Detect* tasks[1]. When provided with CWE, agents complete $19,605 worth of *Detect* tasks. As there are fewer than 1,000 CWEs as of writing, the

[1]$7,920 worth of the detected bounties were disclosed publicly past the model's knowledge cutoff date.

Detect with CWE can be seen analogous to a form of test-time compute scaling, suggesting a path to increasing agent impact. Overall though, while this analysis provides a sense of agent impact on bug bounty programs, it does not account for potential harm caused from cyberattacks via *Exploit*, which is harder to quantify. See Appendix E for more details.

5 Related Work

Offensive Cybersecurity Benchmarks. There have been numerous efforts to develop offensive cybersecurity benchmarks. Most relevant are benchmarks with CTFs such as Cybench [38], and benchmarks with common vulnerabilities and exposures (CVEs) such as CVE-Bench [39], which is concurrent work. In contrast to BountyBench, which covers both offense and defense in a single set of systems and allows us to assess the offense-defense balance, these works are focused exclusively on the offensive cybersecurity setting. Cybench drove significant innovation which we built upon, including task verifiability and real-world metrics. However, the key limitation is that CTFs are not real-world tasks, despite occasionally containing CVEs. CVE-Bench, which also drew inspiration from Cybench, focuses on CVEs in real-world web applications. Whereas CVE-Bench focuses on CVEs with high severity, we focus on a carefully selected subset of bug bounties that are especially meaningful with economic impact. Furthermore, CVE-Bench exclusively focuses on web applications, while BountyBench covers a wider range of settings beyond just web servers, including directly interfacing with libraries. Also, they cover only 8 attack types, whereas our setup supports any number of attack types, and we cover 27 CWEs which span 9 of the OWASP Top 10 Risks. Given the task complexity, they verify each task, which takes 5-24 hours per task. This is helpful, however, the benchmark still lacks task verifiability, where external parties can easily verify that each task is solvable and buildable; in contrast, each task in BountyBench is verified and verifiable. Finally, the works have considerably different setups. CVE-Bench focuses on individual vulnerabilities in single snapshots, and does not provide the codebase at the given commit despite focusing on open-source projects. BountyBench focuses on evolving real-world systems, and each system contains multiple commits and vulnerabilities, all of which can be leveraged to ensure that the task environment replicates the actual setting in which cybersecurity experts operate. Overall though, these efforts are all complementary and help improve understanding of offensive cybersecurity capabilities.

Code Patch Benchmarks. There have been various efforts to develop code patch benchmarks. In particular, SWE-Bench has been popular for evaluating agent performance on resolving GitHub issues; however, this is focused on general software development rather than cybersecurity [20]. There are also concurrent works, such as AutoPatchBench, which is more focused on cybersecurity [24]. AutoPatchBench is focused exclusively on C/C++ vulnerabilities identified through fuzzing and focuses on crash resolution; in contrast, BountyBench focuses more broadly on real-world systems and runs invariant tests including health checks and unit tests to ensure that patches are valid in addition to the exploit. Additionally, these efforts are exclusively focused on patching, whereas BountyBench covers both offense/defense in a single set of systems. Altogether though, these are complementary efforts in this broad space and each provides additional information to better understand the code patching capabilities of AI.

6 Discussion

Limitations and Future Work. While the current benchmark tracks system evolution in a fixed window, to track system evolution into the future, we need to continue to add new vulnerabilities as they are disclosed. Additionally, given the complexity of the system, the evaluators are not absolute. Although the conceptual basis of the *Detect Indicator* is robust, BountyBench is limited to vulnerabilities that have been added to the system. Additionally, agent-written patches may break other parts of the code or not fully resolve the vulnerability because of limitations in human-written invariants and exploits. Here, increasing the number and quality of code invariants, runtime invariants, and exploits could increase confidence. The root cause of the above limitations is that adding systems and tasks is heavily manual work, taking up to tens of hours each.

To mitigate these issues, we want to explore automating task and system creation, and potentially increase the number of gold-standard exploits, patches, and invariants to increase evaluation confidence. In fact, AI agents already exhibit the capability to automate tasks: the *Exploit* task and the *Patch* task mimic the work needed to add new tasks to a given system, i.e. writing an exploit and

patch script to demonstrate solvability. The key challenge is verification to ensure that such tasks are high quality and useful.

Additionally, we focus on evaluating terminal and coding agents, and would like to explore how browser use and other custom tools affect agent performance in future work.

Ethics Statement. Cybersecurity agents are dual-use, capable of supporting both attackers and defenders. We follow the line of researchers who have chosen to release their work publicly and echo the reasoning conveyed in the Ethics Statement in Cybench [38]. In particular: (1) offensive agents are dual use, seen as either a hacking tool for attackers or a pentesting tool for defenders, (2) marginal increase in risk is minimal given other released works in the space, (3) evidence is necessary for informed regulatory decisions and the work helps provide such evidence, and (4) reproducibility and transparency are crucial. We have been heartened to have seen Cybench provide an empirical basis for the AI Safety Institute [33], Anthropic [3], and others in considering AI safety, and hope that BountyBench can help continue this tradition. Finally, unlike Cybench and related works, we also focus on patching vulnerabilities, which favors defenders, and hope to help accelerate this line of research to improve system safety and security.

7 Conclusion

Here we have introduced the first framework to capture offensive and defensive cyber-capabilities in evolving real-world systems. We instantiate this with BountyBench, a benchmark with 25 systems with complex, real-world codebases, and include 40 bug bounties that cover 9 of the OWASP Top 10 Risks. We devise a new *Detect Indicator* for more localized evaluation and comprehensive coverage, and a new strategy to modulate task difficulty based on information. We find that while detecting a zero day remains challenging, agents have strong performance in exploiting and patching vulnerabilities. As the impact of AI agents in cybersecurity grows, it becomes increasingly necessary to thoughtfully evaluate the capabilities and risks of these agents to help guide policy and decision-making. Having designed a framework and instantiated a benchmark to address this need, we plan to continue to update and improve on this work by adding more systems, agents, and tasks.

Acknowledgments

We thank Adam Lambert, Claire Ni, Caroline Van, Hugo Yuwono, Mark Athiri, Alex Yansouni, Zane Sabbagh, Harshvardhan Agarwal, Mac Ya, Fan Nie, Varun Agarwal, Ethan Boyers, and Hannah Kim for their help in reviewing aspects of this work. We thank Open Philanthropy for providing funding for this work. We greatly appreciate huntr and HackerOne and the bug bounty hunters for publicly releasing their bounty reports. We greatly appreciate Alibaba DAMO Academy, the Astropy Project, Benoit Chesneau, BentoML, binary-husky, Composio, the cURL Project, Django Software Foundation, DMLC, Eemeli Aro, Gradio, Invoke, Ionică Bizău, Jason R. Coombs, LangChain, LibreChat, Lightning AI, Lunary, the MLflow Project, the OpenJS Foundation, Python Packaging Authority (PyPA), QuantumBlack, Sebastián Ramírez, scikit-learn, and the vLLM project for releasing their codebases open-source.

References

[1] M. AI. The llama 4 herd: The beginning of a new era of natively multimodal models. `https://ai.meta.com/blog/llama-4-multimodal-intelligence/`, 2025. Accessed: 2025-07-15.

[2] Anthropic. Tools Available to Claude. `https://docs.anthropic.com/en/docs/claude-code/security`.

[3] Anthropic. Claude 3.7 Sonnet System Card. `https://assets.anthropic.com/m/785e231869ea8b3b/original/claude-3-7-sonnet-system-card.pdf`, 2025.

[4] Anthropic. Claude Code Overview. `https://docs.anthropic.com/en/docs/claude-code/overview`, February 2025.